



Indian Institute of Information Technology Kottayam

Department of Computer Science and Engineering

CSE311 – Artificial Intelligence

B.Tech Project Report

VISION-BASED DRONE DETECTION AND LONG-RANGE DISTANCE ESTIMATION

Abstract

This project presents a vision-based approach for detecting drones and estimating their distance from monocular images. The implementation combines a YOLO-based bounding-box representation with a deep-learning regression model based on a pretrained ResNet backbone. The system processes drone images, extracts bounding-box information, crops the detected drone region, and combines visual and geometric features to estimate distance.

The distance-estimation pipeline is implemented using PyTorch and is organised into separate modules for dataset indexing, dataset processing, model definition, training, and evaluation. The system uses the LRDDv3 dataset, whose metadata provides ground-truth three-dimensional distance values. The implementation also supports alternative network configurations and an auxiliary CNN architecture called DroneRanger.

The main experimental configuration uses a ResNet-18 backbone, crop-only processing, six bounding-box features, a bounding-box gating mechanism, Huber loss, a batch size of 32, a learning rate of 0.001, and 50 training epochs. The trained model is evaluated using MAE, Relative MAE, MSE, RMSE, and R^2 .

Contents

1	Introduction	iii
2	Background	iv
2.1	Monocular Drone Distance Estimation	iv
2.2	YOLO Bounding Boxes	iv
2.3	Deep Visual Feature Extraction	iv
2.4	Regression	v
3	Analysis and Requirements	vi
3.1	Functional Requirements	vi
3.2	Dataset Requirements	vi
3.3	Model Requirements	vi
4	Implementation	vii
4.1	Implementation Overview	vii
4.2	Dataset Indexing – <code>index_dataset.py</code>	vii
4.2.1	Functions and Their Responsibilities	viii
4.2.2	Important Code Blocks	viii
4.2.3	Complete Implementation Code	ix
4.2.4	Role in the Complete System	xi
4.3	Dataset Processing – <code>src/dataset.py</code>	xii
4.3.1	Functions and Their Responsibilities	xii
4.3.2	Bounding-Box Conversion	xiv
4.3.3	Bounding-Box Feature Extraction	xiv
4.3.4	Image Preprocessing	xv
4.3.5	Return Values	xv
4.3.6	Complete Implementation Code	xv
4.3.7	Data Processing Flow	xxi
4.4	Model Architecture – <code>src/model.py</code>	xxi
4.4.1	Functions and Their Responsibilities	xxii
4.4.2	Complete Implementation Code	xxiv
4.4.3	Model Flow	xxix
4.5	Model Training – <code>train.py</code>	xxx
4.5.1	Functions and Their Responsibilities	xxx
4.5.2	Complete Implementation Code	xxxiii
4.5.3	Training Flow	xl
4.5.4	Training Configuration Used in the Experiment	xl
4.6	Model Evaluation – <code>evaluate.py</code>	xl

4.6.1	Functions and Their Responsibilities	xli
4.6.2	Complete Implementation Code	xliii
4.7	Experiment Notebook Configuration	li
4.8	Implementation Summary	liii
5	Evaluation	lv
5.1	Evaluation Metrics	lv
5.1.1	Mean Absolute Error	lv
5.1.2	Mean Squared Error	lv
5.1.3	Root Mean Squared Error	lv
5.1.4	R^2	lv
5.2	Evaluation Procedure	lvi
5.3	Evaluation Results	lvi
5.4	Training Curves	lvi
6	Conclusion	lvii
	Appendices	lviii
A	Project File Structure	lviii
B	Complete Experimental Configuration	lix

1 | Introduction

The rapid growth of unmanned aerial vehicles has increased the need for reliable systems capable of detecting drones and estimating their distance from a camera. Such a system can be useful in applications including airspace monitoring, surveillance, safety, and autonomous response.

Traditional distance estimation systems may rely on specialised sensors such as stereo cameras, LiDAR, radar, or depth cameras. However, these sensors can increase the cost and complexity of the system. This project investigates monocular vision as an alternative, using visual information and object bounding-box geometry to estimate the distance of a drone.

The implementation is based on a modular deep-learning pipeline. The input images are associated with YOLO-format bounding-box annotations and metadata containing ground-truth three-dimensional distance. The dataset is indexed and validated before samples are loaded into PyTorch. The drone region is cropped, bounding-box features are calculated, and a neural network predicts the distance.

The implementation is divided into the following major stages:

1. Dataset indexing.
2. Dataset validation and preprocessing.
3. Bounding-box feature extraction.
4. Visual feature extraction using ResNet.
5. Bounding-box feature gating and feature fusion.
6. Distance regression.
7. Model training and validation.
8. Evaluation on unseen test data.

2 | Background

2.1 Monocular Drone Distance Estimation

Monocular distance estimation attempts to infer the distance of an object using a single RGB image. Since a monocular image does not directly provide depth, the model must learn relationships between visual appearance, object scale, image position, and physical distance.

For the drone-distance task, the bounding box provides useful geometric information. A nearby drone is generally represented by a larger bounding box than a distant drone, although the relationship also depends on camera parameters, drone size, orientation, and image content.

2.2 YOLO Bounding Boxes

The implementation uses YOLO-format annotations. A bounding box is represented using normalized coordinates:

$$(x_c, y_c, w, h)$$

where x_c and y_c represent the normalized centre of the bounding box and w and h represent its normalized width and height.

The implementation additionally derives:

$$A = wh$$

and

$$R = \frac{w}{h + \epsilon}$$

where A is the normalized bounding-box area, R is the aspect ratio, and ϵ is a small value used to avoid division by zero.

When the XY feature option is enabled, the complete feature vector is:

$$[x_c, y_c, w, h, A, R].$$

2.3 Deep Visual Feature Extraction

A pretrained ResNet architecture is used as the visual feature extractor. The final classification layer is removed so that the network produces a feature representation instead of an ImageNet class prediction.

The implementation supports ResNet-18, ResNet-50, and ResNet-101. The selected experiment uses ResNet-18, whose feature representation contains 512 values.

2.4 Regression

The final task is a regression problem. Instead of predicting a class, the model produces a single numerical value corresponding to the estimated drone distance.

3 | Analysis and Requirements

3.1 Functional Requirements

The implementation must:

- Locate valid flight sequences in the dataset.
- Match each flight with its correct metadata CSV.
- Verify that required image and label directories exist.
- Remove invalid training samples.
- Read YOLO bounding-box annotations.
- Generate drone image crops.
- Calculate bounding-box features.
- Load and preprocess images.
- Train a neural network for distance regression.
- Save the latest and best model checkpoints.
- Evaluate the trained model on unseen test data.
- Support single-image prediction.

3.2 Dataset Requirements

Each usable flight requires:

- an image directory,
- a YOLO label directory,
- a matching metadata CSV,
- valid image names in the metadata,
- valid values in the `distance_3d_ft` column.

3.3 Model Requirements

The model must accept the drone crop and bounding-box information and produce one distance value. The selected experimental configuration uses six bounding-box features and a ResNet-18 visual backbone.

4 | Implementation

4.1 Implementation Overview

The implementation is organised into independent Python modules. The complete pipeline is:

Dataset → Indexing → Data Processing → Feature Extraction → DistancePredictor → Training → Evaluation.

The implementation consists primarily of:

- `index_dataset.py` – dataset indexing and flight validation.
- `src/dataset.py` – image, annotation, and metadata processing.
- `src/model.py` – distance prediction architectures.
- `train.py` – model training and validation.
- `evaluate.py` – test and single-image evaluation.
- `drone_range.ipynb` – experiment configuration, training, plotting, and evaluation.

4.2 Dataset Indexing – `index_dataset.py`

The `index_dataset.py` module is responsible for scanning the LRDDv3 dataset and identifying valid flight sequences. It searches the dataset for date-wise folders and flight folders, verifies the presence of image and label directories, and finds the metadata CSV corresponding to each flight.

The module supports flight directories with the following prefixes:

- DJI_*
- IMG_*
- MAX_*
- PXL_*

For each flight, the program checks whether an `images` directory and either a `labels` or `final_labels` directory are available. It then searches for the metadata CSV using the exact date and flight ID. Only flights satisfying all these conditions are included in the final dataset manifest.

The `scan_dataset()` function returns a list of dictionaries containing the date, flight ID, image directory, label directory, and metadata CSV path. This manifest is later used by `dataset.py` to load the actual training samples.

4.2.1 Functions and Their Responsibilities

_find_csv_for_flight() This function finds the metadata CSV file corresponding to a particular flight.

It receives:

- `metadata_root` – location of the metadata files.
- `date_str` – date of the flight.
- `flight_id` – unique flight identifier.

The function creates a filename prefix using the date and flight ID and searches for CSV files ending with `_metadata.csv`. Only a CSV whose filename starts with the exact date and flight ID is accepted.

If exactly one matching CSV is found, its path is returned. If multiple files match or no file is found, the function returns `None`.

scan_dataset() This is the main dataset-indexing function.

It performs the following operations:

1. Checks whether the metadata directory exists.
2. Scans the dataset root for date folders.
3. Identifies flight directories using the supported flight patterns.
4. Checks for the `images` directory.
5. Checks for either `labels` or `final_labels`.
6. Finds the metadata CSV corresponding to the exact flight.
7. Adds valid flights to the dataset manifest.

The function returns dictionaries containing:

- `date`
- `flight_id`
- `img_dir`
- `label_dir`
- `csv_path`

4.2.2 Important Code Blocks

```
for entry in sorted(os.listdir(root)):
    date_path = os.path.join(root, entry)
    if not os.path.isdir(date_path):
        continue
    if entry.count("-") != 2:
        continue
```

Listing 4.1: Finding date folders.

This ensures that only folders following the expected date format are processed.

```
for pat in flight_patterns:
    flight_dirs.extend(
        d for d in glob.glob(os.path.join(date_path, pat))
        if os.path.isdir(d))
```

```
)
```

Listing 4.2: Finding flight folders.

This searches for valid flight directories such as DJI_* and MAX_*.

```
manifest.append({
    "date": date_str,
    "flight_id": flight_id,
    "img_dir": img_dir,
    "label_dir": label_dir,
    "csv_path": csv_path,
})
```

Listing 4.3: Creating the dataset manifest.

This stores the validated information required by the next stage of the pipeline.

4.2.3 Complete Implementation Code

```
import os
import glob
from typing import List, Dict, Optional

def _find_csv_for_flight(
    metadata_root: str,
    date_str: str,
    flight_id: str
) -> Optional[str]:
    """
    Enforce date + flight_id match in the CSV name.

    Only accept CSVs that START WITH "{date_str}_{flight_id}_".
    Example valid:
        08-08-2025_MAX_0014_metadata.csv
    """
    wanted_prefix = f"{date_str}_{flight_id}_"

    candidates = glob.glob(os.path.join(metadata_root, "*_metadata.csv"))

    exact = [
        c for c in candidates
        if os.path.basename(c).startswith(wanted_prefix)
    ]

    if len(exact) == 1:
        return exact[0]

    if len(exact) > 1:
        print(
            f"[scan_dataset] !! {date_str}/{flight_id}: "
            f"multiple CSVs matched {wanted_prefix}, "
```

```

        f"candidates={ [os.path.basename(x) for x in exact] } -> skip"
    )
    return None

return None

def scan_dataset(
    root: str,
    metadata_dir: str = "metadata",
    flight_patterns: List[str] = ("DJI_*", "IMG_*", "MAX_*", "PXL_*")
) -> List[Dict]:
    """
    Walk the dataset root and build a manifest of usable flights.

    Returns a list of dicts:
    {
        "date": "04-18-2025",
        "flight_id": "DJI_0080",
        "img_dir": ".../04-18-2025/DJI_0080/images",
        "label_dir": ".../04-18-2025/DJI_0080/labels",
        "csv_path": ".../metadata/04-18-2025_DJI_0080_metadata.csv"
    }
    """

    manifest: List[Dict] = []

    metadata_root = metadata_dir
    if not os.path.isdir(metadata_root):
        raise RuntimeError(
            f"metadata folder not found at {metadata_root}"
        )

    for entry in sorted(os.listdir(root)):
        date_path = os.path.join(root, entry)

        if not os.path.isdir(date_path):
            continue

        if entry.count("-") != 2:
            continue

        date_str = entry

        flight_dirs = []
        for pat in flight_patterns:
            flight_dirs.extend(
                d for d in glob.glob(os.path.join(date_path, pat))
                if os.path.isdir(d)
            )

        if not flight_dirs:
            print(
                f"[scan_dataset] note: no flights found under {date_str}"
            )
            continue

```

```

for flight_dir in sorted(flight_dirs):
    flight_id = os.path.basename(flight_dir)

    img_dir = os.path.join(flight_dir, "images")
    if not os.path.isdir(img_dir):
        print(
            f"[scan_dataset] !! {date_str}/{flight_id}: "
            f"missing images/ at {img_dir} -> skip"
        )
        continue

    label_dir = None
    for cand in ("labels", "final_labels"):
        cand_dir = os.path.join(flight_dir, cand)
        if os.path.isdir(cand_dir):
            label_dir = cand_dir
            break

    if label_dir is None:
        print(
            f"[scan_dataset] !! {date_str}/{flight_id}: "
            "no labels or final_labels -> skip"
        )
        continue

    csv_path = _find_csv_for_flight(
        metadata_root,
        date_str,
        flight_id
    )

    if csv_path is None:
        print(
            f"[scan_dataset] !! {date_str}/{flight_id}: "
            "no matching CSV (strict) -> skip"
        )
        continue

    manifest.append({
        "date": date_str,
        "flight_id": flight_id,
        "img_dir": img_dir,
        "label_dir": label_dir,
        "csv_path": csv_path,
    })

return manifest

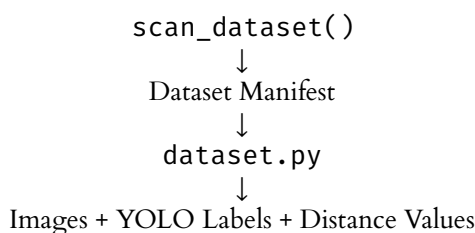
```

Listing 4.4: Complete implementation of `index_dataset.py`.

4.2.4 Role in the Complete System

The output of `index_dataset.py` is used by `dataset.py`:

`index_dataset.py`
 ↓



4.3 Dataset Processing – `src/dataset.py`

The `dataset.py` module is responsible for loading and preprocessing the LRDDv3 dataset. It uses the dataset manifest generated by `index_dataset.py` to locate the images, YOLO label files, and metadata CSV files.

For each image, the module reads the corresponding YOLO bounding-box annotation and extracts the drone region from the image. It also calculates bounding-box features such as width, height, area, and aspect ratio. When `xy_features=True`, the x-coordinate and y-coordinate of the bounding-box centre are also included.

The module reads the ground-truth distance from the `distance_3d_ft` column of the metadata CSV. Invalid samples, missing images, missing distance values, and samples exceeding the specified maximum distance are removed.

The processed data can be returned in two forms:

- **Crop-only mode:** drone crop, bounding-box features, and distance.
- **Full-image mode:** full image, drone crop, bounding-box features, and distance.

The images are resized and normalized using ImageNet mean and standard deviation values before being passed to the neural network.

4.3.1 Functions and Their Responsibilities

`getLRDDDataLoader()` This function creates the PyTorch `DataLoader` used during training, validation, and testing.

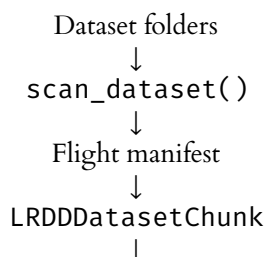
It first calls:

```
manifest = scan_dataset(data_root, metadata_dir)
```

For every flight in the manifest, it creates an `LRDDDatasetChunk` object. All flight datasets are then combined using `ConcatDataset`.

Finally, a PyTorch `DataLoader` is created using the specified batch size, number of workers, and shuffle option.

The data flow is:



ConcatDataset



DataLoader

LRDDDatasetChunk.__init__() The constructor initializes a dataset for one flight.

It:

1. Stores the image and label directories.
2. Reads the metadata CSV using Pandas.
3. Checks whether each image exists.
4. Checks whether each distance value is valid.
5. Removes samples whose distance exceeds `max_dist`.
6. Creates image preprocessing transformations.

The metadata must contain:

- `img_name`
- `distance_3d_ft`

valid_distance() This nested function verifies whether a distance value is valid.

It rejects:

- Missing values.
- NaN values.
- Non-numeric values.
- Infinite values.

This prevents invalid distance labels from entering the training dataset.

```
__len__(.
def __len__(self):
    return len(self.metadata)
```

This returns the number of valid samples remaining in the dataset.

__getitem__() This is the main sample-loading function.

For a given index, it:

1. Retrieves the image name from the metadata.
2. Opens the RGB image.
3. Finds the corresponding YOLO label file.
4. Reads the bounding-box coordinates.
5. Converts normalized YOLO coordinates into pixel coordinates.
6. Crops the drone from the image.
7. Calculates bounding-box features.
8. Resizes and normalizes the crop.
9. Reads the ground-truth distance.
10. Returns the tensors required by the model.

4.3.2 Bounding-Box Conversion

YOLO stores bounding boxes as normalized values:

$$x_{\text{center}}, \quad y_{\text{center}}, \quad w, \quad h.$$

The code converts these values into pixel coordinates:

$$x_{\text{center,px}} = x_{\text{center}}W$$

$$y_{\text{center,px}} = y_{\text{center}}H$$

$$w_{\text{px}} = wW$$

$$h_{\text{px}} = hH.$$

The crop boundaries are then calculated:

$$x_{\min} = x_{\text{center,px}} - \frac{w_{\text{px}}}{2}$$

$$y_{\min} = y_{\text{center,px}} - \frac{h_{\text{px}}}{2}$$

$$x_{\max} = x_{\text{center,px}} + \frac{w_{\text{px}}}{2}$$

$$y_{\max} = y_{\text{center,px}} + \frac{h_{\text{px}}}{2}.$$

The drone crop is obtained using:

```
crop = img.crop((x_min, y_min, x_max, y_max))
```

4.3.3 Bounding-Box Feature Extraction

The code calculates:

$$area = width \times height$$

and

$$aspect = \frac{width}{height + 10^{-6}}.$$

When `xy_features=True`, six features are returned:

$$[x_{\text{center}}, y_{\text{center}}, \text{width}, \text{height}, \text{area}, \text{aspect}].$$

Otherwise, four features are used:

$$[\text{width}, \text{height}, \text{area}, \text{aspect}].$$

4.3.4 Image Preprocessing

The images are:

1. Resized.
2. Converted to tensors.
3. Converted to floating-point values.
4. Normalized using:

$$\text{Mean} = [0.485, 0.456, 0.406]$$

$$\text{Std} = [0.229, 0.224, 0.225].$$

4.3.5 Return Values

In crop-only mode:

```
return crop_tensor, bbox_features, distance
```

Therefore, the model receives:

$$\begin{array}{c} \text{Drone crop} \\ + \\ \text{Bounding-box features} \\ + \\ \text{Ground-truth distance} \end{array}$$

4.3.6 Complete Implementation Code

```
import os
import pandas as pd
from PIL import Image
import torch
import numpy as np
from torch.utils.data import Dataset, DataLoader
import torch.utils.data as data
from torchvision.transforms import v2

from src.index_dataset import scan_dataset

def getLRDDDataLoader(
    data_root,
    metadata_dir,
```

```

        crop_only,
        batch_size,
        num_workers,
        crop_size,
        max_dist=10000,
        shuffle=True,
        is_log=False,
        xy_features=False
    ):

        manifest = scan_dataset(data_root, metadata_dir)
        manifest_df = pd.DataFrame(manifest)

        list_of_datasets = []

        for _, row in manifest_df.iterrows():
            try:
                list_of_datasets.append(
                    LRDDDatasetChunk(
                        row["csv_path"],
                        row["img_dir"],
                        row["label_dir"],
                        crop_only,
                        max_dist,
                        crop_size,
                        transform=None,
                        is_log=is_log,
                        xy_features=xy_features
                    )
                )
            except Exception as e:
                print(
                    f"[extract:ERROR] "
                    f"{row['date']}/{row['flight_id']} failed: {e}"
                )

        full_dataset = data.ConcatDataset(list_of_datasets)

        loader = DataLoader(
            dataset=full_dataset,
            batch_size=batch_size,
            num_workers=num_workers,
            shuffle=shuffle
        )

        return loader

class LRDDDatasetChunk(Dataset):
    """
    Dataset for a single flight.

    Inputs:
        csv_file: path to metadata CSV
        img_folder: directory with RGB frames
        labels_folder: directory with YOLO txt labels
        transform: optional torchvision transform
    """

```

```

__getitem__ returns:
    full_img_tensor
    crop_tensor
    bbox_features
    distance
"""

def __init__(
    self,
    csv_file,
    img_folder,
    labels_folder,
    crop_only,
    max_dist,
    crop_size=512,
    transform=None,
    is_log=False,
    xy_features=False
):

    self.imgs = img_folder
    self.yolo_labels = labels_folder
    self.crop_only = crop_only
    self.is_log = is_log
    self.xy_features = xy_features

    df = pd.read_csv(csv_file)

    keep_mask = []
    missing_image_count = 0
    missing_distance_count = 0

    for _, row in df.iterrows():

        img_name = row["img_name"]
        img_path = os.path.join(self.imgs, img_name)

        if not os.path.exists(img_path):
            missing_image_count += 1
            keep_mask.append(False)
            continue

        raw_dist = row.get("distance_3d_ft", None)

        def valid_distance(val):

            if val is None or pd.isna(val):
                return False

            try:
                float_val = float(val)
            except (TypeError, ValueError):
                return False

            if not np.isfinite(float_val):
                return False

```

```

        return True

    if not valid_distance(raw_dist):
        missing_distance_count += 1
        keep_mask.append(False)
        continue

    elif raw_dist > max_dist:
        keep_mask.append(False)
        continue

    keep_mask.append(True)

df_clean = df[keep_mask].reset_index(drop=True)

self.metadata = df_clean

dropped_total = (
    missing_image_count +
    missing_distance_count
)

if dropped_total > 0:

    if missing_image_count > 0:
        print(
            f"[LRDDDataset] WARN: "
            f"{missing_image_count} rows dropped "
            f"because image file was missing in "
            f"{self.imgs}"
        )

    if missing_distance_count > 0:
        print(
            f"[LRDDDataset] WARN: "
            f"{missing_distance_count} rows dropped "
            f"because distance_3d_ft was missing/invalid "
            f"in {os.path.basename(csv_file)}"
        )

    print(
        f"[LRDDDataset] INFO: kept "
        f"{len(df_clean)} / {len(df)} rows "
        f"for {os.path.basename(csv_file)}"
    )

self.metadata = df_clean
self.yolo_labels = labels_folder

transform = v2.Compose([
    v2.Resize((512, 512)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

```

    )
])

self.full_transform = transform

transform = v2.Compose([
    v2.Resize((crop_size, crop_size)),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

self.crop_transform = transform

def __len__(self):
    return len(self.metadata)

def __getitem__(self, idx):
    row = self.metadata.iloc[idx]
    img_name = row["img_name"]

    img_path = os.path.join(self.imgs, img_name)

    img = Image.open(img_path).convert("RGB")
    img_width, img_height = img.size

    label_path = os.path.join(
        self.yolo_labels,
        os.path.splitext(img_name)[0] + ".txt"
    )

    if os.path.exists(label_path):
        with open(label_path, "r") as f:
            line = f.readline().strip()

        if not line:
            crop = img
            width = height = area = aspect = 0.0
            x_center = y_center = 0.0
        else:
            parts = line.split()

            _, x_center, y_center, width, height = map(
                float,
                parts
            )

            x_center_px = x_center * img_width
            y_center_px = y_center * img_height

```

```

        w_px = width * img_width
        h_px = height * img_height

        x_min = x_center_px - w_px / 2
        y_min = y_center_px - h_px / 2
        x_max = x_center_px + w_px / 2
        y_max = y_center_px + h_px / 2

        crop = img.crop(
            (x_min, y_min, x_max, y_max)
        )

        area = width * height
        aspect = width / (height + 1e-6)

    else:

        crop = img
        width = height = area = aspect = 0.0
        x_center = y_center = 0.0

    if not self.crop_only:
        full_img_tensor = self.full_transform(img)

    crop_tensor = self.crop_transform(crop)

    distance = torch.tensor(
        row["distance_3d_ft"],
        dtype=torch.float32
    )

    if self.xy_features:
        bbox_features = torch.tensor(
            [
                x_center,
                y_center,
                width,
                height,
                area,
                aspect
            ],
            dtype=torch.float32
        )
    else:
        bbox_features = torch.tensor(
            [
                width,
                height,
                area,
                aspect
            ],
            dtype=torch.float32
        )

    if self.is_log:
        distance = np.log(distance + 1e-6)

```

```

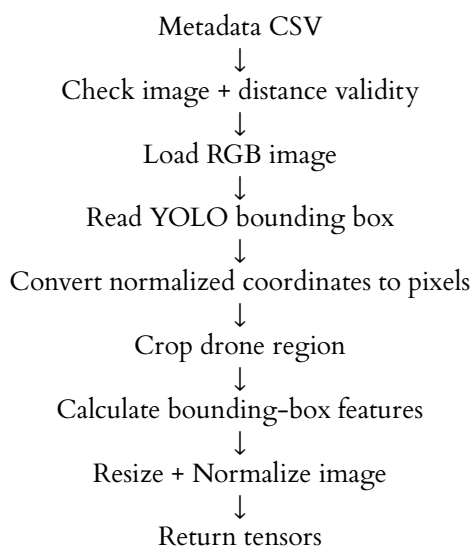
if self.crop_only:
    return crop_tensor, bbox_features, distance

else:
    return (
        full_img_tensor,
        crop_tensor,
        bbox_features,
        distance
    )

```

Listing 4.5: Complete implementation of `src/dataset.py`.

4.3.7 Data Processing Flow



The configuration used in the experiment is:

- `crop_only` = True
- `xy_features` = True
- `bbox_feat_dim` = 6

Thus, the model receives:

Drone crop
+
[x_{center} , y_{center} , *width*, *height*, *area*, *aspect*]
+
Ground-truth distance

The ground-truth distance is used during training as the regression target.

4.4 Model Architecture – `src/model.py`

The `model.py` module contains the neural-network architectures used for drone distance estimation. Two models are implemented: `DistancePredictor` and `DroneRanger`.

The `DistancePredictor` uses pretrained ResNet backbones to extract visual features from the drone image crop. These visual features are combined with bounding-box features to predict the distance of the drone.

The model supports ResNet-18, ResNet-50, and ResNet-101 backbones. Different prediction heads can also be selected using the `alt_head` parameter.

For the configuration used in the proposed implementation, the model uses:

- Backbone: ResNet-18
- Crop-only mode: Enabled
- Bounding-box features: 6
- Alternative head: 6
- Bounding-box gate: Enabled

The six bounding-box features are:

- x-center
- y-center
- width
- height
- area
- aspect ratio

The bounding-box gate transforms these six features into 512 gating values using a fully connected layer and sigmoid activation. These values are multiplied with the 512-dimensional visual features obtained from ResNet-18. The resulting features are then combined with the original bounding-box features and passed through the regression head to produce a single distance value.

4.4.1 Functions and Their Responsibilities

`DistancePredictor.__init__()` This constructor creates the main distance prediction architecture.

It performs four main tasks:

1. Selects the ResNet backbone.
2. Removes the original classification layer.
3. Creates the optional bounding-box gate.
4. Creates the final regression head.

The supported backbones are:

- ResNet-18
- ResNet-50
- ResNet-101

For the experiment, ResNet-18 is selected.

The original ResNet classification layer is removed using:

```
self.backbone_crop.fc = nn.Identity()
```

This allows ResNet to act as a feature extractor rather than an image classifier.

Bounding-Box Gate For `alt_head=6`, the model creates:

```
self.bbox_gate = nn.Sequential(
    nn.Linear(bbox_feat_dim, backbone_out_dim),
    nn.Sigmoid()
)
```

The six bounding-box features are converted into 512 gating values.

These gating values control the contribution of the extracted visual features:

```
gate = self.bbox_gate(bbox_features)
crop_feat = crop_feat * gate
```

This allows the bounding-box information to influence the visual representation.

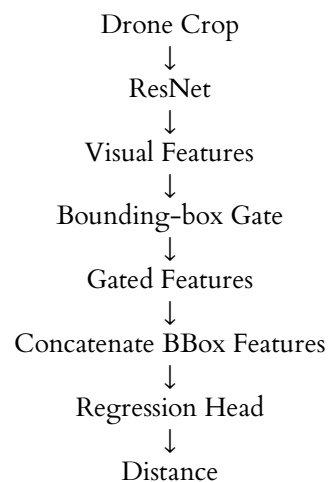
Prediction Head For `alt_head=6`, the regression head is:

```
self.head = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(combined_dim, 256),
    nn.ReLU(),
    nn.Linear(256, 1)
)
```

The final layer produces one numerical value representing the predicted drone distance.

DistancePredictor.forward() The `forward()` function defines how input data passes through the network.

In crop-only mode:



The function returns:

```
return out.squeeze(1)
```

which produces one distance prediction for each sample.

DroneRanger.__init__() DroneRanger is an alternative CNN-based distance estimation architecture.

Instead of using ResNet, it uses several convolutional layers followed by batch normalization, ReLU activation, and average pooling.

The extracted features are combined with bounding-box features and passed to a regression head.

DroneRanger.forward() This function:

1. Extracts CNN features from the crop.
2. Flattens the features.
3. Concatenates them with the bounding-box features.
4. Passes them through the regression head.
5. Returns the predicted distance.

Note: The experiment configuration uses DistancePredictor, not DroneRanger.

4.4.2 Complete Implementation Code

```
import torch
import torch.nn as nn
import torchvision.models as models

class DistancePredictor(nn.Module):

    def __init__(
        self,
        backbone_name="resnet18",
        alt_head=0,
        crop_only=True,
        bbox_feat_dim=4
    ):

        super().__init__()

        self.crop_only = crop_only
        self.alt_head = alt_head

        if backbone_name == "resnet18":

            self.backbone_full = models.resnet18(
                weights=models.ResNet18_Weights.DEFAULT
            )

            self.backbone_crop = models.resnet18(
                weights=models.ResNet18_Weights.DEFAULT
            )

            backbone_out_dim = 512

        elif backbone_name == "resnet50":

            self.backbone_full = models.resnet50(
                weights=models.ResNet50_Weights.DEFAULT
```

```

    )

    self.backbone_crop = models.resnet50(
        weights=models.ResNet50_Weights.DEFAULT
    )

    backbone_out_dim = 2048

elif backbone_name == "resnet101":

    self.backbone_full = models.resnet101(
        weights=models.ResNet101_Weights.DEFAULT
    )

    self.backbone_crop = models.resnet101(
        weights=models.ResNet101_Weights.DEFAULT
    )

    backbone_out_dim = 2048

else:
    raise ValueError(
        "Invalid backbone:",
        backbone_name
    )

# Remove final classification layer
self.backbone_full.fc = nn.Identity()
self.backbone_crop.fc = nn.Identity()

if crop_only:
    combined_dim = backbone_out_dim + bbox_feat_dim
else:
    combined_dim = (
        backbone_out_dim * 2 +
        bbox_feat_dim
    )

if self.alt_head == 4 or \
    self.alt_head == 5 or \
    self.alt_head == 6:

    self.norm = nn.LayerNorm(combined_dim)

if self.alt_head == 5 or self.alt_head == 6:

    self.bbox_gate = nn.Sequential(
        nn.Linear(
            bbox_feat_dim,
            backbone_out_dim
        ),
        nn.Sigmoid()
    )

if alt_head == 1:

    self.head = nn.Sequential(

```

```

        nn.Dropout(0.2),
        nn.Linear(combined_dim, 128),
        nn.ReLU(),
        nn.Dropout(0.2),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, 32),
        nn.ReLU(),
        nn.Linear(32, 1)
    )

elif alt_head == 2:

    self.head = nn.Sequential(
        nn.Dropout(0.2),
        nn.Linear(combined_dim, 128),
        nn.ReLU(),
        nn.Dropout(0.2),
        nn.Linear(128, 32),
        nn.ReLU(),
        nn.Linear(32, 1)
    )

elif alt_head == 3:

    self.head = nn.Sequential(
        nn.Dropout(0.2),
        nn.Linear(combined_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, 1)
    )

elif alt_head == 4:

    self.head = nn.Sequential(
        nn.Dropout(0.2),
        nn.Linear(combined_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 1)
    )

elif alt_head == 5:

    self.head = nn.Sequential(
        nn.Dropout(0.2),
        nn.Linear(combined_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 1)
    )

elif alt_head == 6:

    self.head = nn.Sequential(

```

```

        nn.Dropout(0.2),
        nn.Linear(combined_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 1)
    )

else:

    self.head = nn.Sequential(
        nn.Dropout(0.2),
        nn.Linear(combined_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 1)
    )

def forward(
    self,
    crop_img,
    bbox_features,
    full_img=None
):

    if self.crop_only:

        crop_feat = self.backbone_crop(
            crop_img
        )

        if self.alt_head == 5 or \
            self.alt_head == 6:

            gate = self.bbox_gate(
                bbox_features
            )

            crop_feat = crop_feat * gate

        x = torch.cat(
            [crop_feat, bbox_features],
            dim=1
        )

        if self.alt_head == 4 or \
            self.alt_head == 5:

            x = self.norm(x)

        out = self.head(x)

        return out.squeeze(1)

    else:

        full_feat = self.backbone_full(
            full_img
        )

```

```

        crop_feat = self.backbone_crop(
            crop_img
        )

        x = torch.cat(
            [
                full_feat,
                crop_feat,
                bbox_features
            ],
            dim=1
        )

        out = self.head(x)

        return out.squeeze(1)

class DroneRanger(nn.Module):

    def __init__(self, bbox_feat_dim=4):

        super().__init__()

        self.backbone = nn.Sequential(

            nn.Conv2d(
                3, 8,
                kernel_size=(3, 3),
                bias=False
            ),

            nn.BatchNorm2d(8),
            nn.ReLU(),
            nn.AvgPool2d(2, stride=2),

            nn.Conv2d(
                8, 16,
                kernel_size=(3, 3),
                bias=False
            ),

            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.AvgPool2d(2, stride=2),

            nn.Conv2d(
                16, 32,
                kernel_size=(3, 3),
                bias=False
            ),

            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.AvgPool2d(2, stride=2),

            nn.Conv2d(

```

```

        32, 32,
        kernel_size=(3, 3),
        bias=False
    ),

    nn.BatchNorm2d(32),
    nn.ReLU(),
    nn.AvgPool2d(2, stride=2)
)

backbone_out_dim = 28800

self.head = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(
        backbone_out_dim + bbox_feat_dim,
        256
    ),
    nn.ReLU(),
    nn.Linear(256, 1)
)

def forward(
    self,
    crop_img,
    bbox_features
):
    crop_feat = self.backbone(crop_img)

    x = torch.cat(
        [
            crop_feat.view(
                crop_feat.size(0),
                -1
            ),
            bbox_features
        ],
        dim=1
    )

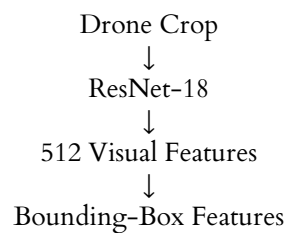
    out = self.head(x)

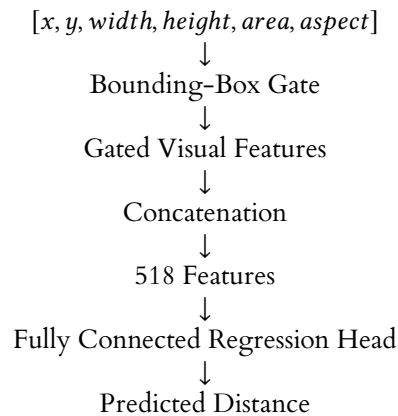
    return out.squeeze(1)

```

Listing 4.6: Complete implementation of `src/model.py`.

4.4.3 Model Flow





4.5 Model Training – **train.py**

The **train.py** module is responsible for training the drone distance estimation model. It creates the training and validation data loaders using `getLRDDDataLoader()`, initializes the selected model architecture, defines the optimization method and loss function, and performs the training process.

The training implementation supports both **DistancePredictor** and **DroneRanger** models. It also supports two loss functions: Mean Squared Error (MSE) and Huber Loss.

During every epoch, the model is trained using the training dataset and then evaluated using the validation dataset. The validation stage calculates MAE and RMSE in addition to the validation loss.

The implementation also uses early stopping. If the validation loss does not improve for a specified number of epochs, training is stopped. The latest model and the best-performing model are saved as `.pth` checkpoint files.

Training metrics are stored in a JSON file containing:

- Epoch
- Training loss
- Validation loss
- Validation MAE
- Validation RMSE

4.5.1 Functions and Their Responsibilities

train() The `train()` function controls the complete model-training process.

It accepts parameters for:

- Dataset location.
- Model backbone.
- Batch size.
- Learning rate.
- Number of epochs.
- Loss function.
- Device.
- Early stopping.

- Bounding-box feature configuration.
- Checkpoint paths.

Creating Training and Validation DataLoaders The function creates separate training and validation loaders:

```
train_loader = getLRDDDataLoader(...)
val_loader = getLRDDDataLoader(...)
```

The training loader provides samples used to update the model, while the validation loader is used to measure performance without updating the model.

Model Initialization The code selects between the two available models:

```
if use_droneranger:
    model = DroneRanger(...)
else:
    model = DistancePredictor(...)
```

For the experiment:

```
use_droneranger = False
```

so `DistancePredictor` is used.

Optimizer The Adam optimizer is created using:

```
optimizer = Adam(model.parameters(), lr=lr)
```

The optimizer updates the model parameters based on the calculated gradients.

Loss Function The implementation supports two loss functions:

```
if use_huber:
    criterion = nn.HuberLoss()
else:
    criterion = nn.MSELoss()
```

The experiment uses Huber Loss.

The loss measures the difference between the predicted distance and the ground-truth distance.

Training Loop The training loop runs for the specified number of epochs:

```
for epoch in range(epochs):
```

For every batch:

1. Input tensors are moved to the selected device.

2. Gradients are reset.
3. The model produces a prediction.
4. The loss is calculated.
5. Backpropagation calculates gradients.
6. The optimizer updates the model.

The important operations are:

```
optimizer.zero_grad()
pred = model(crop_img, bbox_feat)
loss = criterion(pred, distance)
loss.backward()
optimizer.step()
```

Validation After training for an epoch, the model is switched to evaluation mode:

```
model.eval()
```

and gradient calculation is disabled:

```
with torch.no_grad():
```

The model generates predictions for the validation dataset.

Validation Metrics The following metrics are calculated:

```
mse = np.mean((preds - targets) ** 2)
mae = np.mean(np.abs(preds - targets))
rmse = np.sqrt(mse)
```

They represent:

- **MSE:** Mean Squared Error.
- **MAE:** Mean Absolute Error.
- **RMSE:** Root Mean Squared Error.

Model Checkpointing The latest model is saved using:

```
torch.save(
    model.state_dict(),
    checkpoint_path + "_latest.pth"
)
```

If validation loss improves, the best model is saved:

```
torch.save(
    model.state_dict(),
    checkpoint_path + "_best.pth"
)
```

Early Stopping The variable `patience_counter` keeps track of consecutive epochs without improvement.

When:

```
patience_counter >= patience
```

training stops:

```
print("Early stopping triggered")
break
```

Saving Training Metrics Training and validation metrics are stored in a JSON file:

```
with open(metrics_path, "w") as f:
    json.dump(metrics, f, indent=4)
```

This allows the training curves to be plotted later from the notebook.

4.5.2 Complete Implementation Code

```
import torch
from torch.utils.data import DataLoader
from torch.optim import Adam
import torch.nn as nn
import numpy as np
from tqdm import tqdm
import json

from src.model import DistancePredictor, DroneRanger
from src.dataset import getLRDDDataLoader

def train(
    data_root="/mnt/active_storage/Knut/LRDD_v3",
    metadata_dir="/mnt/active_storage/Knut/LRDD_v3/metadata",
    backbone="resnet50",
    crop_only=True,
    use_huber=False,
    use_droneranger=False,
    use_alt_head=False,
    epochs=20,
    batch_size=16,
    lr=1e-4,
```

```

device="cuda",
patience=5,
min_delta=1e-4,
checkpoint_path="best_model.pth",
metrics_path="training_metrics.json",
bbox_feature_dim=4,
num_workers=4,
checkpoint_weights=None,
crop_size=512,
is_log=False,
xy_features=False
):

train_data_root = data_root + "/train"
train_metadata_dir = metadata_dir + "/train"

val_data_root = data_root + "/val"
val_metadata_dir = metadata_dir + "/val"

train_loader = getLRDDDataLoader(
    train_data_root,
    train_metadata_dir,
    crop_only,
    batch_size,
    num_workers=num_workers,
    crop_size=crop_size,
    is_log=is_log,
    xy_features=xy_features
)

val_loader = getLRDDDataLoader(
    val_data_root,
    val_metadata_dir,
    crop_only,
    batch_size,
    num_workers=num_workers,
    crop_size=crop_size,
    is_log=is_log,
    xy_features=xy_features
)

if use_droneranger:

    model = DroneRanger(
        bbox_feat_dim=bbox_feature_dim
    ).to(device)

else:

    model = DistancePredictor(
        backbone_name=backbone,
        alt_head=use_alt_head,
        crop_only=crop_only,
        bbox_feat_dim=bbox_feature_dim,
    ).to(device)

if checkpoint_weights is not None:

```

```

        model.load_state_dict(
            torch.load(
                checkpoint_weights,
                map_location=device
            )
        )

    optimizer = Adam(
        model.parameters(),
        lr=lr
    )

    if use_huber:
        criterion = nn.HuberLoss()
    else:
        criterion = nn.MSELoss()

    best_val_loss = np.inf
    patience_counter = 0

    metrics = {
        "epoch": [],
        "train_loss": [],
        "val_loss": [],
        "val_mae": [],
        "val_rmse": []
    }

    for epoch in range(epochs):

        print(
            f"\nEpoch {epoch + 1}/{epochs}"
        )

        model.train()

        train_loss = 0

        pbar = tqdm(
            train_loader,
            desc="Training",
            leave=False
        )

        if crop_only:

            for crop_img, bbox_feat, distance in pbar:

                crop_img = crop_img.to(device)
                bbox_feat = bbox_feat.to(device)
                distance = distance.to(device).float()

                optimizer.zero_grad()

                pred = model(
                    crop_img,

```

```

        bbox_feat
    )

    loss = criterion(
        pred,
        distance
    )

    loss.backward()
    optimizer.step()

    batch_loss = loss.item()

    train_loss += batch_loss

    pbar.set_postfix(
        {"batch_loss": batch_loss}
    )
else:
    for (
        full_img,
        crop_img,
        bbox_feat,
        distance
    ) in pbar:

        full_img = full_img.to(device)
        crop_img = crop_img.to(device)
        bbox_feat = bbox_feat.to(device)
        distance = distance.to(device).float()

        optimizer.zero_grad()

        pred = model(
            crop_img,
            bbox_feat,
            full_img
        )

        loss = criterion(
            pred,
            distance
        )

        loss.backward()
        optimizer.step()

        batch_loss = loss.item()

        train_loss += batch_loss

        pbar.set_postfix(
            {"batch_loss": batch_loss}
        )

```

```

train_loss /= len(
    train_loader.dataset
)

model.eval()

val_loss = 0

pbar_val = tqdm(
    val_loader,
    desc="Validation",
    leave=False
)

preds = []
targets = []

with torch.no_grad():
    if crop_only:
        for (
            crop_img,
            bbox_feat,
            distance
        ) in pbar_val:

            crop_img = crop_img.to(device)
            bbox_feat = bbox_feat.to(device)
            distance = distance.to(device).float()

            pred = model(
                crop_img,
                bbox_feat
            )

            loss = criterion(
                pred,
                distance
            )

            val_loss += loss.item()

            pbar_val.set_postfix(
                {"batch_loss": loss.item()}
            )

            preds.append(
                pred.cpu().numpy()
            )

            targets.append(
                distance.cpu().numpy()
            )
    else:

```

```

    for (
        full_img,
        crop_img,
        bbox_feat,
        distance
    ) in pbar_val:

        full_img = full_img.to(device)
        crop_img = crop_img.to(device)
        bbox_feat = bbox_feat.to(device)
        distance = distance.to(device).float()

        pred = model(
            crop_img,
            bbox_feat,
            full_img
        )

        loss = criterion(
            pred,
            distance
        )

        val_loss += loss.item()

        pbar_val.set_postfix(
            {"batch_loss": loss.item()}
        )

        preds.append(
            pred.cpu().numpy()
        )

        targets.append(
            distance.cpu().numpy()
        )

preds = np.concatenate(preds)
targets = np.concatenate(targets)

mse = np.mean(
    (preds - targets) ** 2
)

mae = np.mean(
    np.abs(preds - targets)
)

rmse = np.sqrt(mse)

val_loss /= len(
    val_loader.dataset
)

print(
    f" Train Loss: {train_loss:.4f} | "
    f"Val Loss: {val_loss:.4f} | "

```

```

        f"Val MAE: {mae:.4f} | "
        f"Val RMSE: {rmse:.4f}"
    )

    metrics["epoch"].append(
        epoch + 1
    )

    metrics["train_loss"].append(
        float(train_loss)
    )

    metrics["val_loss"].append(
        float(val_loss)
    )

    metrics["val_mae"].append(
        float(mae)
    )

    metrics["val_rmse"].append(
        float(rmse)
    )

    torch.save(
        model.state_dict(),
        checkpoint_path + "_latest.pth"
    )

    if val_loss < best_val_loss - min_delta:

        best_val_loss = val_loss
        patience_counter = 0

        torch.save(
            model.state_dict(),
            checkpoint_path + "_best.pth"
        )

        print(
            f"Improvement detected. "
            f"Saving model to {checkpoint_path}"
        )

    else:

        patience_counter += 1

        print(
            f" No improvement "
            f"(Patience {patience_counter}/{patience})"
        )

    if patience_counter >= patience:

        print(
            "Early stopping triggered"

```

```

    )

    break

    with open(metrics_path, "w") as f:

        json.dump(
            metrics,
            f,
            indent=4
        )

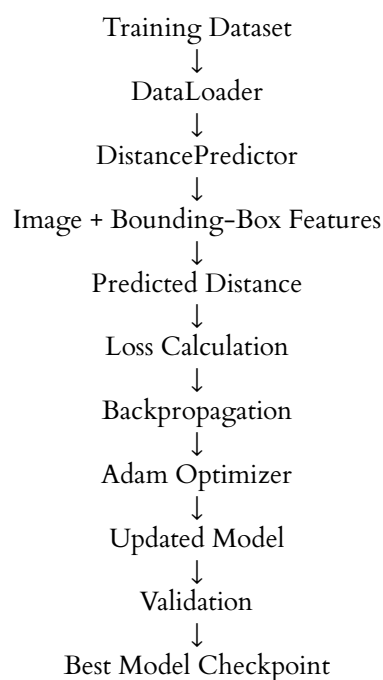
    print(
        f"\nMetrics saved to {metrics_path}"
    )

    return model, metrics

```

Listing 4.7: Complete implementation of `train.py`.

4.5.3 Training Flow



4.5.4 Training Configuration Used in the Experiment

The experiment in `drone_range.ipynb` uses:

4.6 Model Evaluation – `evaluate.py`

The `evaluate.py` module evaluates the trained distance estimation model on unseen test data. It loads the trained model checkpoint, generates distance predictions, compares them with the ground-truth distances, and calculates multiple regression metrics.

Table 4.1: Training configuration used in the experiment.

Parameter	Value
Backbone	ResNet-18
Crop Only	True
Loss	Huber Loss
Alternative Head	6
Epochs	50
Batch Size	32
Learning Rate	0.001
Patience	50
Crop Size	220×220
BBox Features	6
Log Distance	False
XY Features	True

The module contains two main functions:

- `evaluate_model()` – evaluates the model on the complete test dataset.
- `evaluate_model_single()` – performs distance estimation for a single image.

4.6.1 Functions and Their Responsibilities

`evaluate_model()` This function performs batch evaluation on the test dataset.

It first creates the test DataLoader:

```
test_loader = getLRDDDataLoader(...)
```

Then it creates the same model architecture used during training and loads the trained checkpoint:

```
model.load_state_dict(
    torch.load(checkpoint_path, map_location=device)
)
```

The model is switched to evaluation mode:

```
model.eval()
```

For each test sample, the model predicts the drone distance and stores both predictions and ground-truth values.

Prediction and Ground-Truth Collection The function maintains two arrays:

```
preds = []
targets = []
```

Predictions are appended to `preds`, while actual distances are appended to `targets`.

After processing all test samples:

```
preds = np.concatenate(preds)
targets = np.concatenate(targets)
```

creates complete arrays for metric calculation.

Evaluation Metrics The function calculates:

```
mse = np.mean((preds - targets) ** 2)
mae = np.mean(np.abs(preds - targets))
rmse = np.sqrt(mse)
```

It also calculates Relative MAE:

```
rel_mae = np.mean(np.abs(preds / targets))
```

and R^2 :

```
ss_res = np.sum((preds - targets) ** 2)
ss_tot = np.sum((targets - targets.mean()) ** 2)
r2 = 1 - (ss_res / ss_tot)
```

The final metrics are:

- MAE
- Relative MAE
- MSE
- RMSE
- R^2

Saving Predictions If:

```
save_predictions=True
```

the predictions and ground-truth values are saved to:

```
test_predictions.csv
```

using Pandas.

evaluate_model_single() This function evaluates one image.

It:

1. Locates the image.
2. Reads its metadata.
3. Retrieves the ground-truth `distance_3d_ft`.
4. Reads the corresponding YOLO label.
5. Creates the drone crop.
6. Calculates bounding-box features.
7. Preprocesses the image.
8. Loads the trained model.
9. Generates the distance prediction.
10. Returns the image, crop, target distance, and predicted distance.

Single-Image Bounding-Box Processing The function reads the YOLO annotation:

```
parts = line.split()
_, x_center, y_center, width, height = map(
    float,
    parts
)
```

It converts the normalized coordinates into pixel coordinates and crops the drone using:

```
crop = img.crop(
    (x_min, y_min, x_max, y_max)
)
```

It then creates the bounding-box feature vector.

4.6.2 Complete Implementation Code

```
import torch
import numpy as np
from tqdm import tqdm
from PIL import Image
import pandas as pd
import os
from torchvision.transforms import v2

from src.model import DistancePredictor, DroneRanger
from src.dataset import getLRDDDataLoader

@torch.no_grad()
def evaluate_model(
    data_root,
    metadata_dir,
    checkpoint_path,
    backbone="resnet50",
```

```

crop_only=True,
use_droneranger=False,
use_alt_head=0,
batch_size=16,
device="cuda",
save_predictions=False,
bbox_feature_dim=4,
num_workers=4,
max_dist=10000,
crop_size=512,
is_log=False,
xy_features=False
):

test_data_root = data_root + "/test"
test_metadata_dir = metadata_dir + "/test"

test_loader = getLRDDDataLoader(
    test_data_root,
    test_metadata_dir,
    crop_only,
    batch_size,
    num_workers=num_workers,
    crop_size=crop_size,
    max_dist=max_dist,
    is_log=is_log,
    xy_features=xy_features
)

if use_droneranger:

    model = DroneRanger(
        bbox_feat_dim=bbox_feature_dim
    ).to(device)

else:

    model = DistancePredictor(
        backbone_name=backbone,
        alt_head=use_alt_head,
        crop_only=crop_only,
        bbox_feat_dim=bbox_feature_dim,
    ).to(device)

model.load_state_dict(
    torch.load(
        checkpoint_path,
        map_location=device
    )
)

model.eval()

preds = []
targets = []

if crop_only:

```

```

for crop_img, bbox_feat, distance in tqdm(
    test_loader
):

    crop_img = crop_img.to(device)
    bbox_feat = bbox_feat.to(device)
    distance = distance.to(device).float()

    outputs = model(
        crop_img,
        bbox_feat
    )

    preds.append(
        outputs.cpu().numpy()
    )

    targets.append(
        distance.cpu().numpy()
    )

else:

    for (
        full_img,
        crop_img,
        bbox_feat,
        distance
    ) in tqdm(test_loader):

        full_img = full_img.to(device)
        crop_img = crop_img.to(device)
        bbox_feat = bbox_feat.to(device)
        distance = distance.to(device).float()

        outputs = model(
            crop_img,
            bbox_feat,
            full_img
        )

        preds.append(
            outputs.cpu().numpy()
        )

        targets.append(
            distance.cpu().numpy()
        )

preds = np.concatenate(preds)
targets = np.concatenate(targets)

if is_log:

    preds = torch.exp(
        torch.tensor(preds)
    )

```

```

    ).numpy()

    targets = torch.exp(
        torch.tensor(targets)
    ).numpy()

mse = np.mean(
    (preds - targets) ** 2
)

mae = np.mean(
    np.abs(preds - targets)
)

rmse = np.sqrt(mse)

rel_mae = np.mean(
    np.abs(preds / targets)
)

ss_res = np.sum(
    (preds - targets) ** 2
)

ss_tot = np.sum(
    (targets - targets.mean()) ** 2
)

r2 = 1 - (ss_res / ss_tot)

print("\n==== Test Metrics =====")

print(f"MAE : {mae:.4f}")
print(f"Rel MAE : {rel_mae:.4f}")
print(f"MSE : {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R : {r2:.4f}")

if save_predictions:

    df = pd.DataFrame({
        "target": targets,
        "prediction": preds
    })

    df.to_csv(
        "test_predictions.csv",
        index=False
    )

    print(
        "\nPredictions saved to test_predictions.csv"
    )

return (
    preds,
    targets,

```

```

        {
            "mae": mae,
            "rel": rel_mae,
            "mse": mse,
            "rmse": rmse,
            "r2": r2
        }
    )

@torch.no_grad()
def evaluate_model_single(
    data_folder,
    metadata_file,
    image_name,
    checkpoint_path,
    backbone="resnet50",
    crop_only=True,
    use_droneranger=False,
    use_alt_head=0,
    batch_size=16,
    device="cuda",
    bbox_feature_dim=4,
    crop_size=512
):

    img_folder = data_folder + "images/"
    yolo_labels = data_folder + "labels/"

    df = pd.read_csv(metadata_file)

    row = df[
        df["img_name"] == image_name
    ]

    img_path = os.path.join(
        img_folder,
        image_name
    )

    print(img_path)

    if not os.path.exists(img_path):

        print("Image path not valid")
        return

    raw_dist = row[
        "distance_3d_ft"
    ].iloc[0]

    print(raw_dist)

    def valid_distance(val):

        if val is None or pd.isna(val):
            return False

```

```

    try:
        float_val = float(val)

    except (TypeError, ValueError):
        return False

    if not np.isfinite(float_val):
        return False

    return True

if not valid_distance(raw_dist):

    print(
        "Image does not have a valid distance"
    )

    return

full_transform = v2.Compose([
    v2.Resize((512, 512)),
    v2.ToImage(),
    v2.ToDtype(
        torch.float32,
        scale=True
    ),
    v2.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

crop_transform = v2.Compose([
    v2.Resize((crop_size, crop_size)),
    v2.ToImage(),
    v2.ToDtype(
        torch.float32,
        scale=True
    ),
    v2.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

img = Image.open(
    img_path
).convert("RGB")

img_width, img_height = img.size

label_path = os.path.join(
    yolo_labels,
    os.path.splitext(image_name)[0] + ".txt"
)

```

```

if os.path.exists(label_path):

    with open(label_path, "r") as f:
        line = f.readline().strip()

    if not line:

        crop = img

        width = height = 0.0
        area = aspect = 0.0

    else:

        parts = line.split()

        _, x_center, y_center, width, height = map(
            float,
            parts
        )

        x_center_px = (
            x_center * img_width
        )

        y_center_px = (
            y_center * img_height
        )

        w_px = width * img_width
        h_px = height * img_height

        x_min = (
            x_center_px -
            w_px / 2
        )

        y_min = (
            y_center_px -
            h_px / 2
        )

        x_max = (
            x_center_px +
            w_px / 2
        )

        y_max = (
            y_center_px +
            h_px / 2
        )

        crop = img.crop(
            (
                x_min,
                y_min,
                x_max,

```

```

        y_max
    )
)

area = width * height

aspect = width / (
    height + 1e-6
)

else:

    crop = img

    width = height = 0.0
    area = aspect = 0.0

if not crop_only:

    full_img_tensor = full_transform(
        img
    )

    crop_tensor = crop_transform(
        crop
    )

    distance = torch.tensor(
        row["distance_3d_ft"],
        dtype=torch.float32
    )

    bbox_features = torch.tensor(
        [
            width,
            height,
            area,
            aspect
        ],
        dtype=torch.float32
    )

if use_droneranger:

    model = DroneRanger(
        bbox_feat_dim=bbox_feature_dim
    ).to(device)

else:

    model = DistancePredictor(
        backbone_name=backbone,
        alt_head=use_alt_head,
        crop_only=crop_only,
        bbox_feat_dim=bbox_feature_dim,
    ).to(device)

```

```

model.load_state_dict(
    torch.load(
        checkpoint_path,
        map_location=device
    )
)

model.eval()

if crop_only:

    crop_tensor = crop_tensor.to(device)
    bbox_features = bbox_features.to(device)
    distance = distance.to(device).float()

    outputs = model(
        crop_tensor.unsqueeze(0),
        bbox_features.unsqueeze(0)
    )

    pred = outputs.cpu().numpy()
    target = distance.cpu().numpy()

else:

    full_img_tensor = full_img_tensor.to(device)
    crop_tensor = crop_tensor.to(device)
    bbox_features = bbox_features.to(device)
    distance = distance.to(device).float()

    outputs = model(
        crop_tensor,
        bbox_features,
        full_img_tensor
    )

    pred = outputs.cpu().numpy()
    target = distance.cpu().numpy()

return (
    img,
    crop,
    target,
    pred
)

```

Listing 4.8: Complete implementation of `evaluate.py`.

4.7 Experiment Notebook Configuration

The main experiment is executed from `drone_range.ipynb`. The selected configuration is:

```
from train import train
```

```

model_variant = "resnet18"

model, metrics = train(
    data_root="/mnt/active_storage/Knut/LRDD_v3",
    metadata_dir="/mnt/active_storage/Knut/LRDD_v3/metadata",
    backbone=model_variant,
    crop_only=True,
    use_huber=True,
    use_droneranger=False,
    use_alt_head=6,
    epochs=50,
    batch_size=32,
    device="cuda:1",
    lr=1e-3,
    patience=50,
    checkpoint_path=model_variant+"_huber_220_alt6_xy",
    metrics_path=model_variant+"_huber_220_alt6_xy_training_metrics.json",
    bbox_feature_dim=6,
    num_workers=32,
    crop_size=220,
    is_log=False,
    xy_features=True
)

```

Listing 4.9: Main experiment configuration from `drone_range.ipynb`.

The training metrics can subsequently be loaded and plotted:

```

%matplotlib inline

import json
import matplotlib.pyplot as plt

with open(
    "resnet18_huber_220_alt6_xy_training_metrics.json"
) as f:
    metrics = json.load(f)

plt.plot(
    metrics["epoch"],
    metrics["train_loss"],
    label="Train Loss"
)

plt.plot(
    metrics["epoch"],
    metrics["val_loss"],
    label="Val Loss"
)

plt.plot(
    metrics["epoch"],
    metrics["val_mae"],
    label="Val MAE"
)

```

```
plt.plot(
    metrics["epoch"],
    metrics["val_rmse"],
    label="Val RMSE"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

Listing 4.10: Training-curve plotting code.

The trained model is evaluated using:

```
from evaluate import evaluate_model

model_variant = "resnet18"

evaluate_model(
    data_root="/mnt/active_storage/Knut/LRDD_v3",
    metadata_dir="/mnt/active_storage/Knut/LRDD_v3/metadata",
    checkpoint_path=model_variant+"_huber_220_alt6_xy_best.pth",
    backbone=model_variant,
    crop_only=True,
    use_droneranger=False,
    use_alt_head=6,
    batch_size=32,
    device="cuda:1",
    save_predictions=False,
    bbox_feature_dim=6,
    num_workers=32,
    max_dist=10000,
    crop_size=220,
    is_log=False,
    xy_features=True
)
```

Listing 4.11: Model evaluation call used in the experiment.

4.8 Implementation Summary

The implementation establishes a complete modular pipeline for monocular drone-distance estimation. Dataset indexing first identifies valid flight sequences and ensures that image, annotation, and metadata files correspond to the same flight. Dataset processing then validates the samples, converts YOLO annotations into image crops, and creates the geometric feature representation.

The model combines visual features extracted by ResNet-18 with bounding-box information. The selected alternative architecture uses a bounding-box gate to modulate the visual features before regression. Training uses Huber loss and Adam optimisation, while validation tracks MAE and RMSE. Checkpointing and early stopping are used to retain the best-performing model.

Finally, the evaluation module loads the trained checkpoint and measures its predictions against ground-truth distances using several regression metrics. The implementation also provides a single-image inference function for testing the complete processing pipeline on an individual image.

5 | Evaluation

The evaluation of the implementation should establish how accurately the system estimates drone distance on unseen data. The evaluation procedure implemented in `evaluate.py` calculates MAE, Relative MAE, MSE, RMSE, and R^2 .

The evaluation should be presented using numerical evidence from the actual experiment rather than unsupported claims. In particular, the final report should include the metric values printed by the test evaluation and, where available, training and validation curves.

5.1 Evaluation Metrics

5.1.1 Mean Absolute Error

Mean Absolute Error is calculated as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

It represents the average absolute difference between predicted and ground-truth distance.

5.1.2 Mean Squared Error

Mean Squared Error is:

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2.$$

5.1.3 Root Mean Squared Error

Root Mean Squared Error is:

$$RMSE = \sqrt{MSE}.$$

5.1.4 R^2

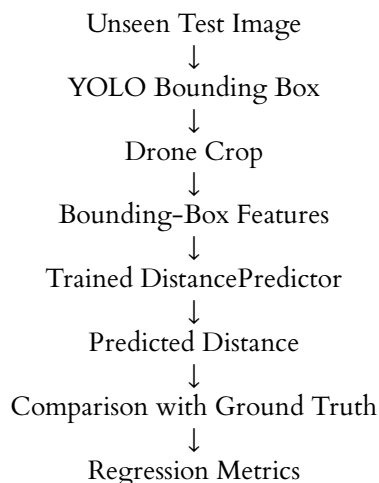
The implementation calculates:

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

5.2 Evaluation Procedure

The test dataset is loaded using the same dataset-processing pipeline used during training. The trained checkpoint is loaded, the model is switched to evaluation mode, and gradients are disabled. Predictions are then compared against the corresponding ground-truth distances.

The evaluation procedure is therefore:



5.3 Evaluation Results

The numerical values in this section should be populated directly from the actual output of `evaluate_model()`. The implementation does not assume or fabricate distance-estimation results.

Table 5.1: Test-set distance estimation metrics.

Metric	Measured Value
MAE	To be filled from experiment
Relative MAE	To be filled from experiment
MSE	To be filled from experiment
RMSE	To be filled from experiment
R^2	To be filled from experiment

5.4 Training Curves

The training notebook stores epoch-wise training loss, validation loss, validation MAE, and validation RMSE in a JSON file. These values can be plotted to examine convergence and possible overfitting.

6 | Conclusion

This project implements a modular vision-based drone distance-estimation pipeline using monocular images, YOLO bounding-box information, and deep visual features. The implementation separates dataset indexing, data preprocessing, model construction, training, and evaluation into independent components.

The dataset-indexing module ensures that only valid flights are used. The data-processing module validates image and distance information, reads YOLO annotations, extracts drone crops, and constructs geometric features. The distance-prediction architecture uses a pretrained ResNet-18 backbone and combines its visual representation with six bounding-box features.

The selected training configuration uses Huber loss, Adam optimisation, 50 epochs, a batch size of 32, a learning rate of 0.001, and a bounding-box gating mechanism. Model checkpoints and training metrics are saved during training, and the evaluation module provides several regression metrics for assessing performance.

Future work can include improving the distance-estimation accuracy, investigating alternative visual backbones, evaluating additional bounding-box representations, incorporating camera calibration information, and testing the model under a wider range of environmental and flight conditions.

A | Project File Structure

The implementation is organised approximately as follows:

```
drone-distance-estimation/  
|  
+-- src/  
| +-- dataset.py  
| +-- index_dataset.py  
| +-- model.py  
|  
+-- train.py  
+-- evaluate.py  
+-- drone_range.ipynb
```

B | Complete Experimental Configuration

The final experimental configuration used for the reported model is:

```
model_variant = "resnet18"

crop_only = True
use_huber = True
use_droneranger = False
use_alt_head = 6

epochs = 50
batch_size = 32
lr = 1e-3
patience = 50

bbox_feature_dim = 6
num_workers = 32
crop_size = 220

is_log = False
xy_features = True
```